APPLICATION RESILIENCE / CONTROLLED EXPERIMENTS

APPLICATION RESILIENCE REPORT

Failure Detection, Recovery Mechanisms & Reliability Validation

REPORT DATE	27 October 2026	**APPLICATION / VERSION**	Not supplied
ENVIRONMENT	Not supplied; isolation not verified	**EVIDENCE**	No source, logs, screenshots or measurements supplied

EXECUTIVE SUMMARY

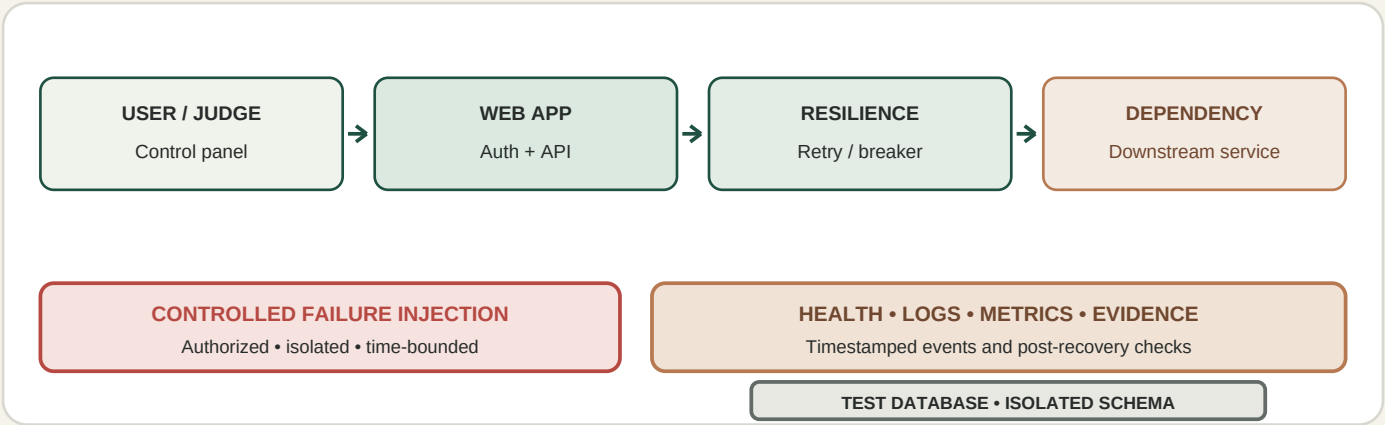
This report defines an evidence-led review of four controlled experiments: a downstream dependency outage, an isolated test-database table failure, an invalid result record, and a workload of exactly 100 requests. It documents what to detect, how to validate recovery, and which evidence is required before any reliability claim is accepted.

Evidence status: NOT VERIFIED / NOT MEASURED. No application artifact or test output was supplied. The architecture below is a logical reference model, not proof of implementation or successful execution.

OBJECTIVE & DECISION RULE

Demonstrate safe failure detection, controlled degradation, data validation, bounded execution, and a verified return to predefined healthy criteria. A fallback may preserve user-facing behavior while a dependency remains down; it is not proof that the dependency recovered. A 100-request run proves only the behavior of that specific test configuration, not production-scale capacity.

LOGICAL SYSTEM ARCHITECTURE



Normal flow	Forest-green arrows represent the expected request path through application resilience controls.
Failure injection	Crimson denotes an authorized, isolated, time-bounded fault; do not target production systems.
Recovery evidence	Copper denotes health checks, timestamped logs, metrics and a real post-recovery request.
Test database	Use a dedicated schema and verify database identity before any destructive fixture operation.

Operating contract

Every experiment must have an owner, test ID, target identity, baseline health criteria, bounded fault/workload settings, a stop condition, cleanup instructions, and a saved evidence bundle. If any safety guard or isolation check fails, do not inject the fault.

01 / FAILURE DETECTION & RECOVERY

Detection is not recovery

Capture a timestamped sequence: injection → first detection → resilience action → restoration or cleanup → passing post-recovery health check. Define the recovery start event in advance; use the same rule across comparable runs.

SCENARIO	DETECTION SIGNAL	RECOVERY / HEALING PATH	REQUIRED EVIDENCE
1. Dependency outage Connection refused, timeout, retry exhaustion or failed probe.	Classify retryable vs. permanent errors; record attempt count, last error and breaker transitions.	Apply bounded retries and backoff; open breaker at configured threshold; use only an explicitly safe fallback. Restore service and verify probe plus a real request.	Injection and detection timestamps; retry log; breaker event; fallback response; dependency probe; post-recovery request.
2. Test-database table failure Missing table, permission or query exception.	Capture safe error class and request/run ID; confirm database and schema identity.	Return controlled error without leaking query details. Restore the table/schema in the isolated test database; verify the intended query and an unrelated operation.	Database/schema identity; sanitized exception; response; restore command/result; two passing queries.
3. Invalid result record Schema, type, required-field or integrity violation.	Record validation reason code and compare valid/rejected counts before and after.	Reject or quarantine per policy. Preserve valid records; avoid silent coercion, partial writes or exposing sensitive internals.	Input fixture; expected validation rule; reason code; rejected count; valid-record read/checksum or query.
4. Exactly 100 requests HTTP status, timeout, latency, throughput and resource metrics.	Reconcile client ledger to exactly 100 launched requests; capture start/end timestamps and terminal status for each.	Use an explicit concurrency limit and timeout. Monitor health during run and after stabilization. Recovery is N/A unless degradation occurred.	Request ledger; endpoint; concurrency and timeout; status counts; latency distribution; post-run health check.

MEASUREMENT DEFINITIONS

METRIC	OPERATIONAL DEFINITION	RECORDING RULE
Detection time	First detected timestamp – injection timestamp.	Use event timestamps from the same clock domain; preserve timezone and precision.
Recovery time	Recovery criteria-met timestamp – explicitly defined recovery-start timestamp.	State whether recovery begins at restore action, retry resume, or another defined event.
Experiment duration	Completion timestamp – experiment start timestamp.	Include execution, cleanup and verification; do not substitute only the injection window.
Load duration	First request start → last request terminal outcome; stabilization measured separately.	Report request duration and post-run health stabilization independently.

COMMON RUN SEQUENCE

STEP	PHASE	REQUIRED ACTION
1	**Baseline**	Capture health endpoint, latency/error baseline, target identity and version.
2	**Inject / execute**	Record exact injection time, configured fault, request bounds and run ID.
3	**Observe**	Capture detection signal, resilience state transitions, response and resource metrics.
4	**Restore / clean up**	Restore dependency or test fixture; stop any workload; verify no lingering fault.
5	**Prove recovery**	Run the same health criteria and a representative real operation; archive evidence.

PASS / FAIL DECISION CRITERIA

SCENARIO	EVIDENCE SUPPORTING PASS	FAIL / INCONCLUSIVE SIGNAL
Dependency outage	Expected resilience state is observed; service is restored; dependency probe and representative real request both pass.	Fallback alone is counted as recovery, no event timeline exists, or the dependency is never rechecked.
Test DB table	Isolated schema is confirmed; controlled error is observed; table restored; intended and unrelated queries pass.	Database identity is unknown, production access is possible, or restore validation is missing.
Invalid record	Input is rejected/quarantined as designed; counts reconcile; valid records remain readable and intact.	Invalid data is silently accepted/coerced, valid records change unexpectedly, or counts do not reconcile.
100 requests	Ledger has exactly 100 unique terminal outcomes; totals reconcile; explicit bounds were used; health passes after stabilization.	Totals are not 100, outcomes are double-counted/missing, run is unbounded, or post-run health fails.

Safety gate: do not run destructive database or dependency experiments without verified target identity, explicit authorization, a bounded window and a tested rollback / cleanup path.

02 / BEFORE–DURING–AFTER EVIDENCE

Measurement ledger

Populate this ledger from actual test output. Do not mark PASS unless the expected behavior and recovery criteria are evidenced. Attach application version, environment, configuration fingerprint, run ID and timestamp to every scenario.

SCENARIO	BEFORE: BASELINE	DURING: FAILURE / RUN	AFTER: RECOVERY	EVIDENCE STATUS
Dependency	Health, latency and error rate: **To be measured**. Dependency identity: not supplied.	Retry count, breaker transitions, fallback result and error classes: **To be measured**.	Dependency probe + real request: **Not verified**; recovery time not measured.	Logs/screenshots not supplied. **PENDING**
Test DB table	DB/schema identity and intended query: **Not verified**.	Exception class and controlled application response: **Not measured**.	Restored query + unrelated operation: **Not verified**.	Isolation proof not supplied. **PENDING**
Invalid record	Valid-record count and integrity baseline: **To be measured**.	Validation failure reason, rejected count and error response: **To be measured**.	Valid records remain accessible: **Not verified**.	Validation logs/counts not supplied. **PENDING**
100 requests	Baseline health and latency: **To be measured**.	Total=100; success/failure counts; average, p50, p95 and optionally p99; RPS and error rate: **To be measured**.	Post-run health: **Not verified**; stabilization not measured.	Request ledger not supplied. **PENDING**

100-REQUEST RECONCILIATION

CONFIGURATION / METRIC	REQUIREMENT / FORMULA	OBSERVED
Total launched requests	Exactly 100 to the designated test endpoint.	Not measured
Scope and bounds	Test endpoint only; explicit concurrency cap, per-request timeout and overall deadline.	Not supplied
Terminal reconciliation	Successful + failed + timed out + cancelled/other terminal outcomes = 100. Define each category so no request is double-counted.	Not measured
Success / error rate	Success rate = successful / 100 × 100%. Error rate = failed / 100 × 100%; separately disclose timeouts and cancellations if they are not included as failures.	Not measured
Latency / throughput	Average, p50 and p95; p99 only with method and sample count. Requests/sec = completed requests / measured run duration.	Not measured
Health after test	Re-run baseline health criteria and representative operation after a stated stabilization interval.	Not verified

Percentile note: a p99 estimate from only 100 observations is sensitive to method and sampling. State whether latency includes client/network overhead, how retries are counted, and whether throughput uses wall-clock or request-active duration. A completed load run is not inherently a recovery event.

EVIDENCE BUNDLE CHECK

MINIMUM RUN RECORD	VALUE TO CAPTURE	CURRENT STATUS
Run ID / version / environment	Unique identifier; commit/build; environment; sanitized configuration fingerprint.	Not supplied
Target and safety controls	Endpoint or service identity; DB/schema identity when relevant; authorization; bounds and stop conditions.	Not verified
Event timeline	Baseline, injection/start, detection, resilience action, restore/cleanup, criteria-met, verification.	Not measured
Attachments	Sanitized logs, request ledger, health output, screenshots or machine-readable test results.	Not supplied

Evidence integrity: all observed-value fields are intentionally left unpopulated. This document is a measurement plan, not proof that experiments passed or that a particular implementation exists.

03 / RELIABILITY ASSESSMENT

Findings, limitations & next steps

Current assessment: not verifiable from supplied artifacts. This is an evidence-status conclusion, not a finding that the application is unreliable. Implementation details and outcomes must be established from source, configuration and repeatable runs.

VERIFICATION MATRIX

CLAIM / CONTROL	STATUS	EVIDENCE REQUIRED TO VERIFY
Retry policy / error classification	**Not verified**	Retryable vs. permanent error rules, maximum attempts, backoff/jitter configuration, and timestamped attempt logs.
Circuit breaker transitions	**Not verified**	Thresholds, open/half-open/closed events, probe outcome and a successful post-recovery request.
Fallback safety / freshness	**Not verified**	Captured fallback output, explicit stale/unavailable labeling, expiry policy and separate proof of dependency recovery.
Test database isolation	**Not verified**	Sanitized connection/configuration details, DB/schema identity, permissions, audit trail and proof no production target was used.
Invalid-record policy	**Not verified**	Validation rules, reason codes, before/after counts, reject/quarantine outcome and successful read of valid records.
Exactly 100 bounded requests	**Not verified**	Client ledger with exactly 100 unique request outcomes, timeout/concurrency settings and reconciled terminal counts.
Recovery and durable evidence	**Not verified**	Timestamped health output, run ID, relevant sanitized logs/metrics, cleanup confirmation and saved report artifact.

PRIORITIZED VALIDATION PLAN

PRIORITY	ACTION	EXIT CRITERION
1 · Safety	Prove chaos/fault injection is authorized, test-only, audited and time-bounded. Block production targets and verify the cleanup/rollback path before execution.	Unauthorized invocation fails closed; target identity and test window are recorded.
2 · Recovery	Run dependency outage and capture retries, breaker state, fallback, service restoration, health probe and real request.	All event timestamps present; fallback behavior and dependency recovery independently verified.
3 · Data integrity	Run isolated missing-table and invalid-record cases; record controlled errors and compare counts before/after.	No production access; controlled errors; valid data remains readable and unchanged as expected.
4 · Bounded load	Send exactly 100 requests to a test endpoint with explicit concurrency and timeout limits. Save request-by-request output.	Terminal outcomes reconcile to 100; metrics computed reproducibly; post-run health passes.

GO / NO-GO JUDGE CHECKLIST

SAFETY & SETUP	DETECTION & RECOVERY	INTEGRITY & EVIDENCE
■ Injection authorized and target isolated ■ Version, environment and run ID recorded ■ Bounds, stop condition and cleanup defined	■ Detection and recovery timestamps captured ■ Retry/breaker/fallback behavior observed ■ Post-recovery health and real request pass	■ Test DB isolation demonstrated ■ Invalid-record policy and counts captured ■ Exactly 100 requests reconciled ■ Logs and results attached / reproducible

LIMITATIONS & FINAL CONCLUSION

No source code, runtime configuration, screenshots, logs, test output, or metrics were provided. Therefore no pass rate, measured recovery time, zero-downtime claim, uptime figure, or production-readiness conclusion is justified. Replace “Not verified” / “Not measured” only with traceable evidence from a real run.

Minimum evidence package: (1) sanitized test configuration and version, (2) test authorization and target identity, (3) timestamped event timeline, (4) scenario-specific logs and output, (5) reconciliation calculations, (6) cleanup confirmation, and (7) post-recovery health evidence.

Reliability claim rule: reproducible failure + observable mechanism + bounded impact + verified recovery + preserved evidence. If any element is missing, report the result as inconclusive rather than PASS.